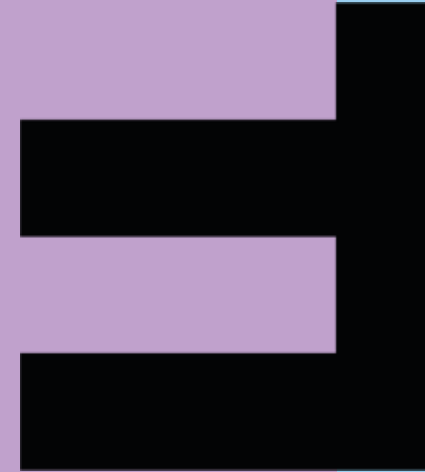


FHV

Vorarlberg University
of Applied Sciences



Technical Information Systems

4. SQL – Introduction

SQL

- Introduction

SQL – brief history

14 November 2023

Seite 3

- Original name: **SEQUEL** - **S**tructured **E**nglish **QU**ery **L**anguage
- First versions in the 70ies at IBM for System R
- 1986: SQL1; SQL (ANSI 1986) or SQL-86
- 1992: SQL2; (supporting “classical” relational databases)
- 1999: SQL3; (supporting object relational databases)
- 2003: adding XML features
- 2006: data mining functions
- 2008: **INSTEAD OF** triggers, **TRUNCATE** statement,[32] **FETCH** clause
- 2011: *SQL:2011* ISO/IEC 9075:2011 – temporal support
- 2016: Row pattern matching, polymorphic table functions, JSON
- 2019: multidimensional arrays
- 2023: Adds data type JSON (not yet in SQL Server)

<https://en.wikipedia.org/wiki/SQL>



- SQL is a standardized Interface language for RDBMS
- SQL is not a full programming language
 - Standalone – with a specialized editor
 - Embedded – as part of a code
- SQL is a **descriptive and declarative** language set:
 - this means:
 - ✧ SQL describes the **result**,
 - ✧ **Not** how the result is to be created



- Caveat:
 - Big database companies extend SQL with some special features and create with this their own database-programming languages (e.g. Oracle - PL/SQL, Microsoft - Transact-SQL)
 - The newest SQL-Standards are so extensive that a specific DBMS typically does not support all features



SQL Environment – Microsoft's usage of Schema

14 November 2023

Seite 6

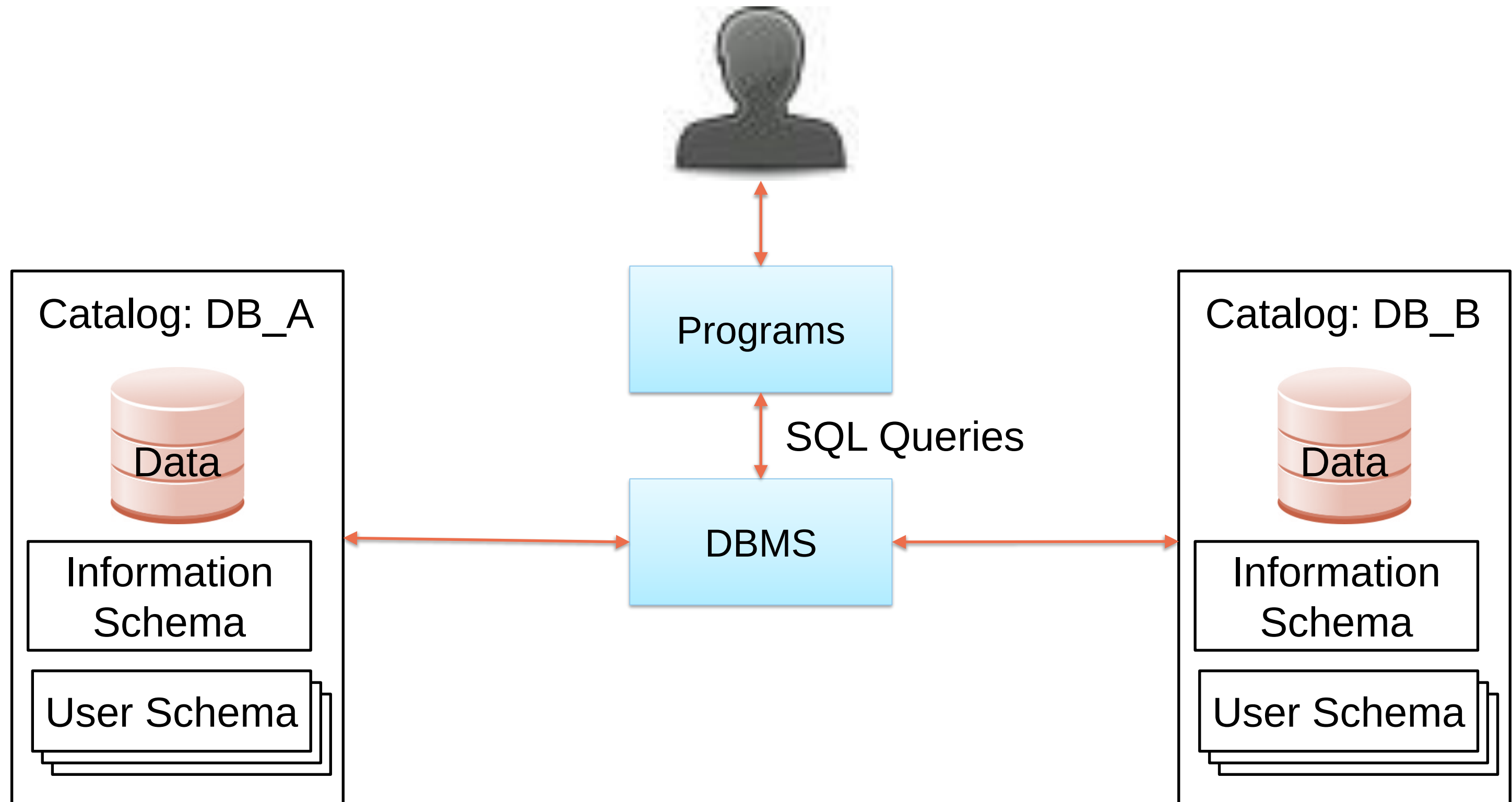
- An SQL **environment** is the installation of a SQL compliant RDBMS in a computer system
- It contains named collections of schemas (also called databases) in MSSQL
- Each catalog has an information schema containing information to the schemas and descriptions in the catalog.
- Schemas are database descriptions
- **Schema elements** include tables, constraints, views, domains, and other constructs that describe the schema.



SQL Environment

14 November 2023

Seite 7



SQL - basic functionality

14 November 2023

Seite 8

Manipulation

DML
Data
Manipulation
Language
Reading,
writing,
replacing,
removing data

Definition

DDL
Data
Definition
Language
Definition,
Modification,
Removal of
data structures

Control

DAL
Data
Administration
Language
Generation,
adaptation,
destruction of
interface objects
(such as users,
files, etc.)

SQL – Datatypes (here T-SQL)

14 November 2023

Seite 9

Exact numerics bigint, bit, decimal, int, money, numeric, smallint, smallmoney, tinyint	Character strings char, varchar, text
Approximate numerics float, real	Unicode character strings nchar, nvarchar, ntext
Date and time date, datetime2, datetime, datetimeoffset, smalldatetime, time	Other data types cursor, rowversion, hierarchyid, uniqueidentifier, sql_variant, xml, Spatial Geometry Types, Spatial Geography Types, table
Binary strings binary, image, varbinary	



Consider the following guidelines to make your T-SQL code easily readable (and therefore easier to understand and debug!):

- Capitalize T-SQL keywords, like SELECT, FROM, AS, and so on. Capitalizing keywords is a commonly used convention that makes it easier to find each clause of a complex statement.
- Start a new line for each major clause of a statement.
- If the SELECT list contains more than a few columns, expressions, or aliases, consider listing each column on its own line.
- Indent lines containing subclauses or columns to make it clear which code belongs to each major clause.

Case-Sensitiveness

- If the SQL statements are case sensitive or not – and which elements of it – depends on the system you are working on.
- Example: <https://dev.mysql.com/doc/refman/8.0/en/identifier-case-sensitivity.html>

→ Stick to case sensitive work



SQL

- Introduction
- Exercises – Setting up the Basis and DDL

Literature

Important Ressources

14 November 2023

Seite 13

- Links to information and references
 - Microsoft SQL Server Management Studio (SSMS) (optional)
<https://docs.microsoft.com/en-us/sql/ssms/sql-server-management-studio-ssms>
 - SQL Server Express Edition or Developer Edition Download, for own Exercises:
 - [SQL Server-Downloads | Microsoft](#)
 - **Azure Data Studio (the IDE we use)**
[Azure Data Studio | Microsoft Azure](#)
[Azure Data Studio FAQ - SQL Server | Microsoft Learn](#)
- T-SQL
<https://learn.microsoft.com/en-us/sql/t-sql/language-reference?view=sql-server-ver15>

Version 16 is the 2022 release



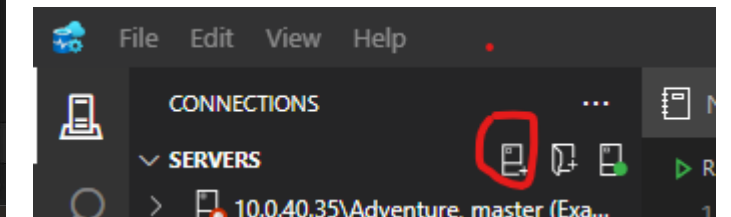
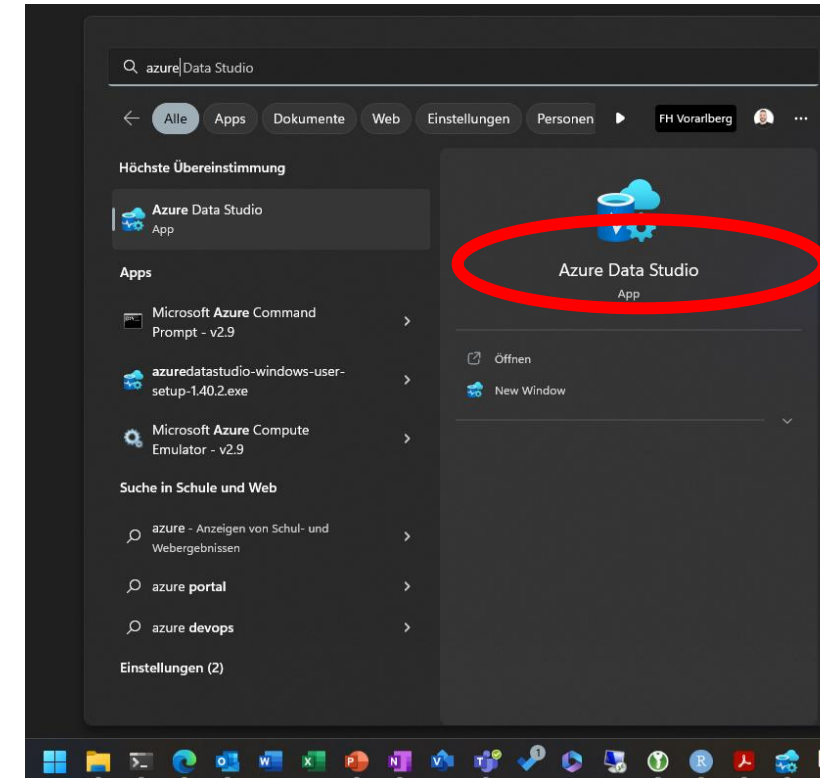
Starting SQL Server and Login with Azure Data Studio

14 November 2023

Seite 14

- We have SQL Server 2017 Standard Edition at server side
- We are working with Azure Data Studio at client side
- Download and install it from here:

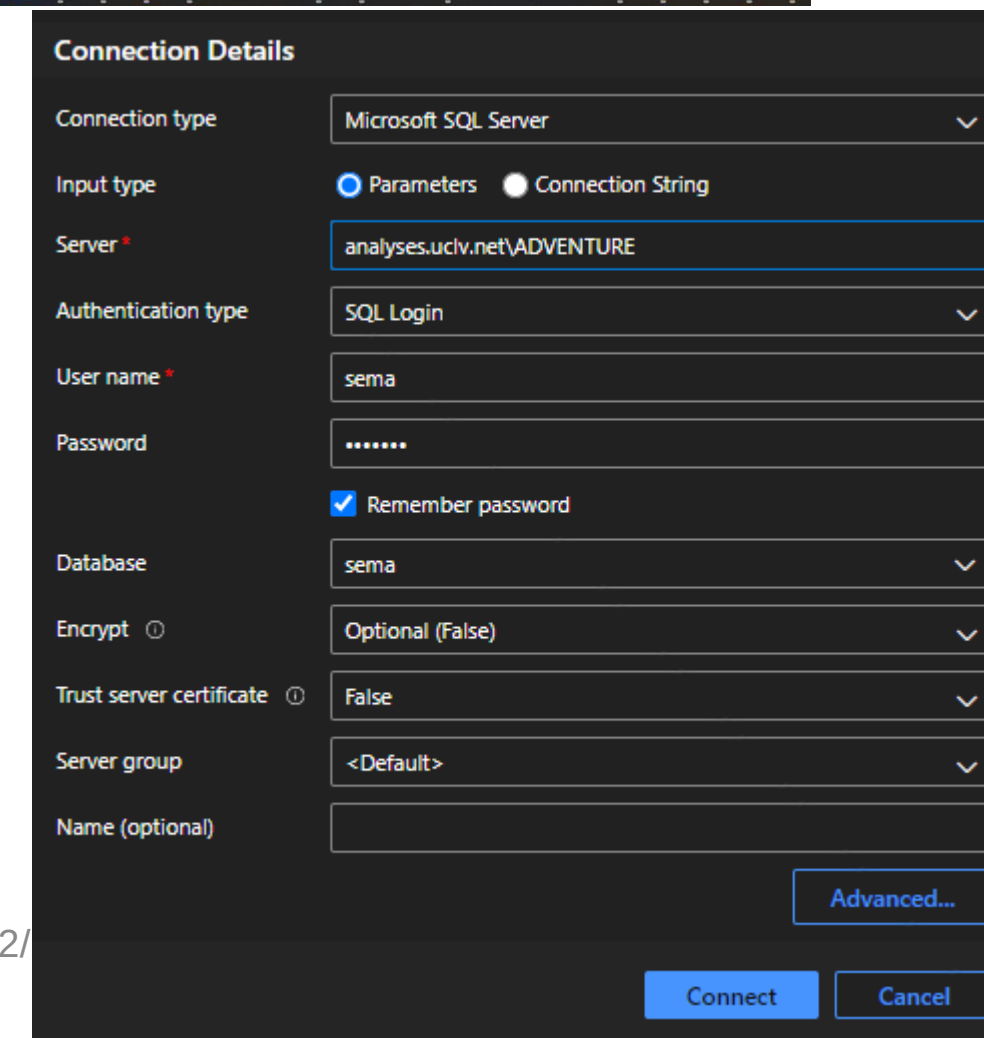
[Download and install Azure Data Studio - Azure Data Studio | Microsoft Learn](#)



On Windows Computers with FHV Login:

- Access to the database (in Azure Data Studio):
File – Connect Object Explorer ...
 - Server Name: analyses.uclv.net\Adventure
 - Authentication: Windows Authentication

Remark: the database works only at the FHV or via VPN



Azure Data Studio VS. Management Studio

14 November 2023

Seite 15

- *Use **Azure Data Studio** if you:*

- Are mostly editing or executing queries.
- Need the ability to quickly chart and visualize result sets.
- Can execute most administrative tasks via the integrated terminal using sqlcmd or PowerShell.
- Have minimal need for wizard experiences.
- Don't need to do deep administrative or platform-related configuration.
- Need to run on macOS or Linux.

- *Use **SQL Server Management Studio** if you:*

- Are doing complex administrative or platform configuration.
- Are doing security management, including user management, vulnerability assessment, and configuration of security features.
- Need to make use of performance tuning advisors and dashboards.
- Use database diagrams and table designers.
- Need access to Registered Servers.
- Make use of live query stats or client statistics.

Source: <https://learn.microsoft.com/en-us/sql/azure-data-studio/faq?view=sql-server-ver15>

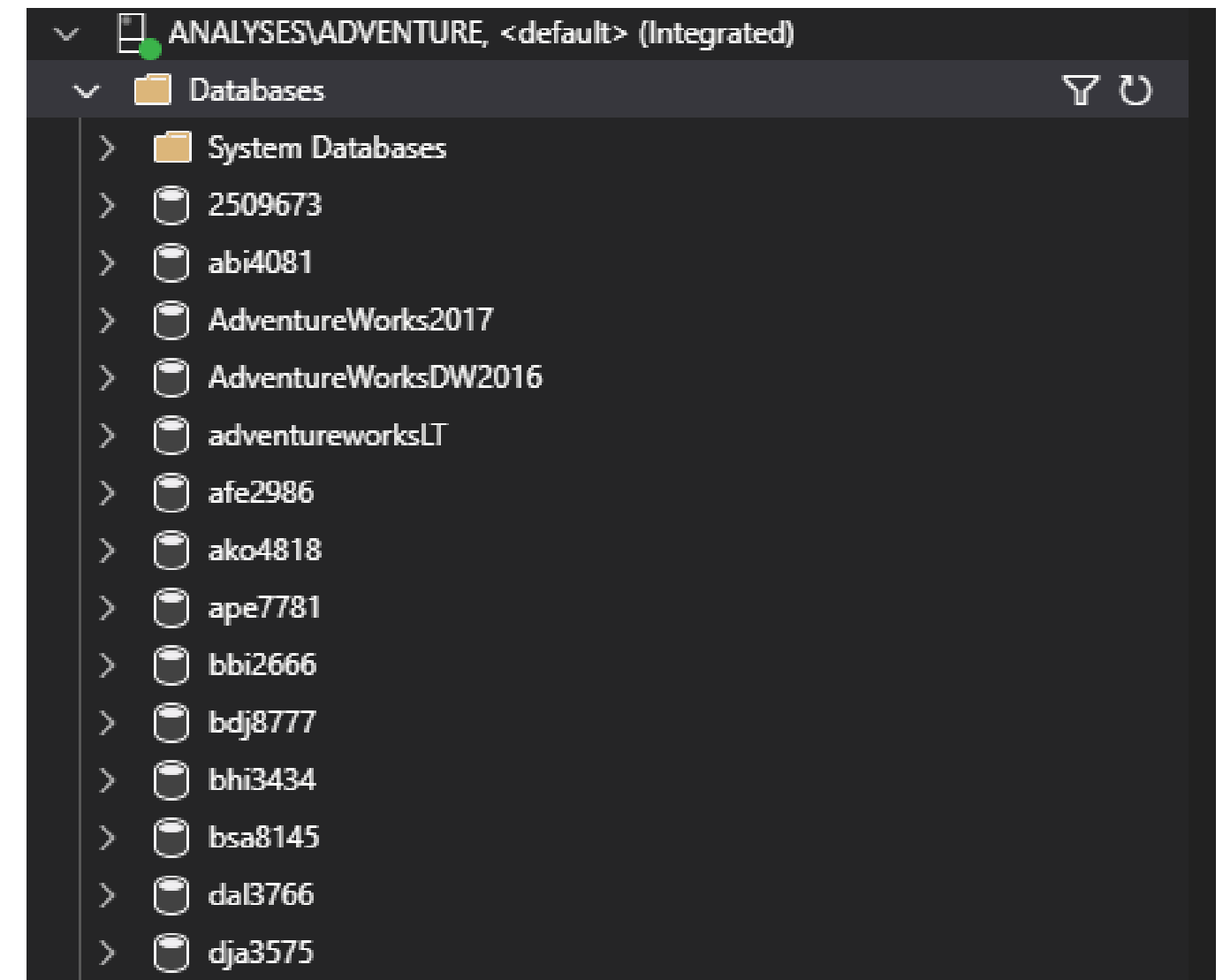


Personal Database

14 November 2023

Seite 16

- You find your own Database Created for you
- The name of your database is the same as your username
- You get a special password for the DB from me
- You have not the rights to create a database on the centralized server
- But you have full db_owner rights in your database
- Further you have db_reader rights on our Training Databases AdventureWorks2017 and AdventureWorksDW2016



<https://learn.microsoft.com/en-us/sql/relational-databases/databases/create-a-database?view=sql-server-ver15>
<https://learn.microsoft.com/en-us/sql/t-sql/statements/create-database-transact-sql?view=sql-server-ver15&tabs=sqlpool>



DDL– Data Definition Language

14 November 2023

Seite 17

- The DDL consists of all commands to define, change and delete the **objects** of a database
- The main commands are
 - CREATE
Does not only create but also define the given object.
e.g. you can create a table and its attributes with one command
 - ALTER
Changes the definitions of an object.
e.g. you can add or delete columns or indexes to a table
 - DROP
Deletes the object
Caveat: There is no undo for that
- T-SQL Reference:
<https://docs.microsoft.com/en-us/sql/t-sql/statements/statements>



DDL – Examples (1)

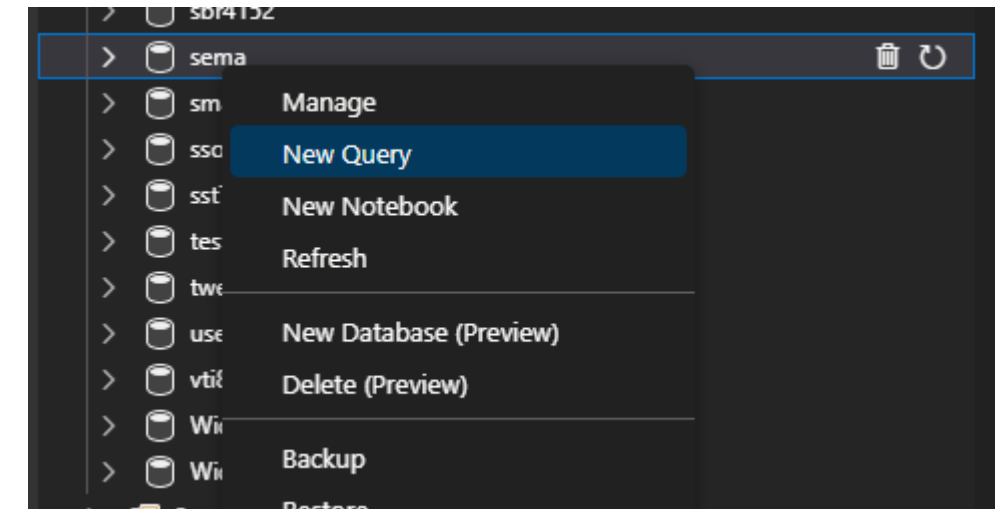
14 November 2023

Seite 18

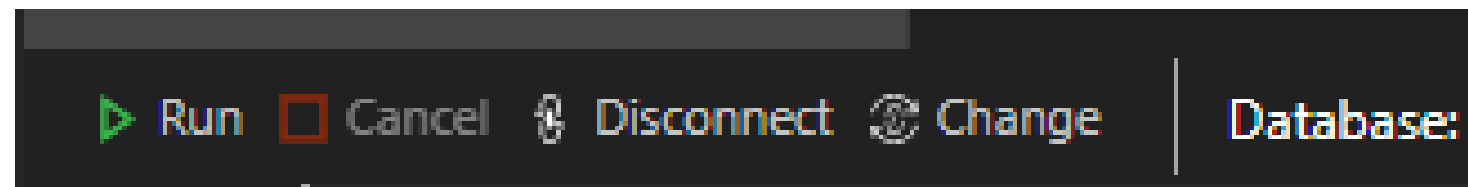
- With your database selected right click on “New Query”

- Code:

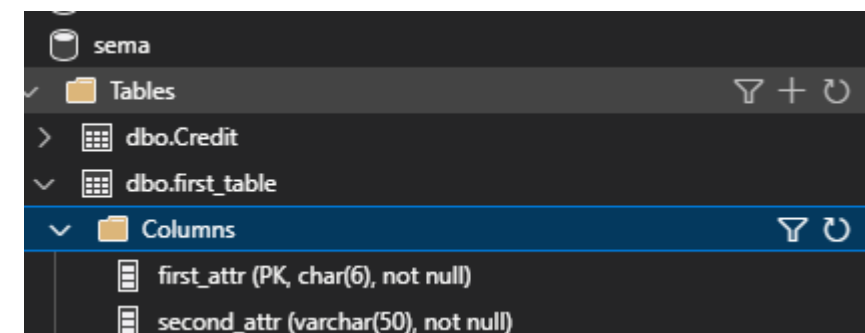
```
CREATE TABLE first_table(  
    first_attr char(6) not null,  
    second_attr varchar(50) not null,  
    CONSTRAINT first_tablePK PRIMARY KEY (first_attr)  
);
```



- Click on “Run”



- Explore your database in the Object Viewer.
You might need to refresh (F5) the view



DDL – Examples (2)

14 November 2023

Seite 19

- With your database selected click on “New Query”

- Add a new attribute

```
ALTER TABLE first_table ADD third_attr INT;
```

- Refresh and explore the database again

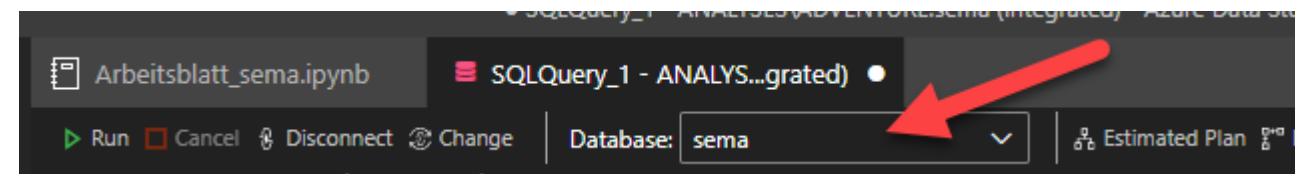
- Now delete the table:

```
DROP TABLE first_table;
```

- Remark: If you fail to start in your database, you can start your statements with

```
USE { database_name }; (e.g. USE [hwe])
```

or check the interface



Importing data

14 November 2023

Seite 20

- There are two files with readymade data for our exercises
“Course_Management_1_create.sql” and “Course_Management_2_data.sql”
- Download both (delete the .sec extension, if existing)
If you use the RDS-version, save them in OneDrive.
- Inspect them in a text editor. What do you see?

- Go to Azure Data Studio
- Make sure to have your database selected
- Open (Ctrl-O) file Course_Management_1_create.sql
- Execute and explore

- Open and execute Course_Management_2_data.sql

- Right-Click on the tables and use “SELECT Top 1000 rows”
to explore the data

Data source and some of the query examples: Michael Mannino, Associate Professor, Business School, University of Colorado Denver

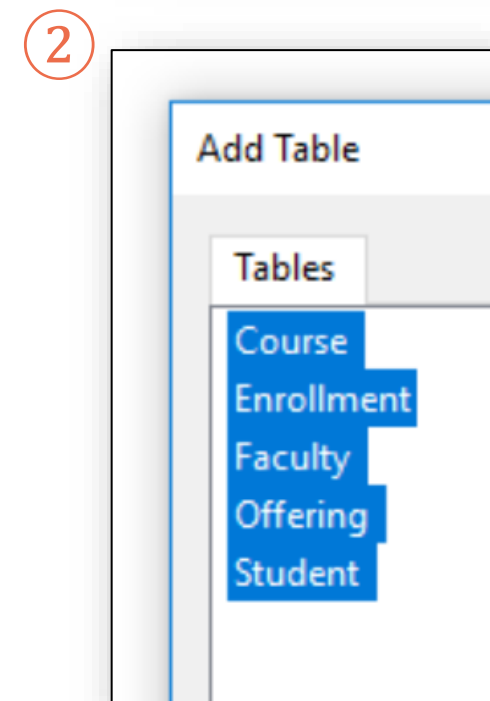
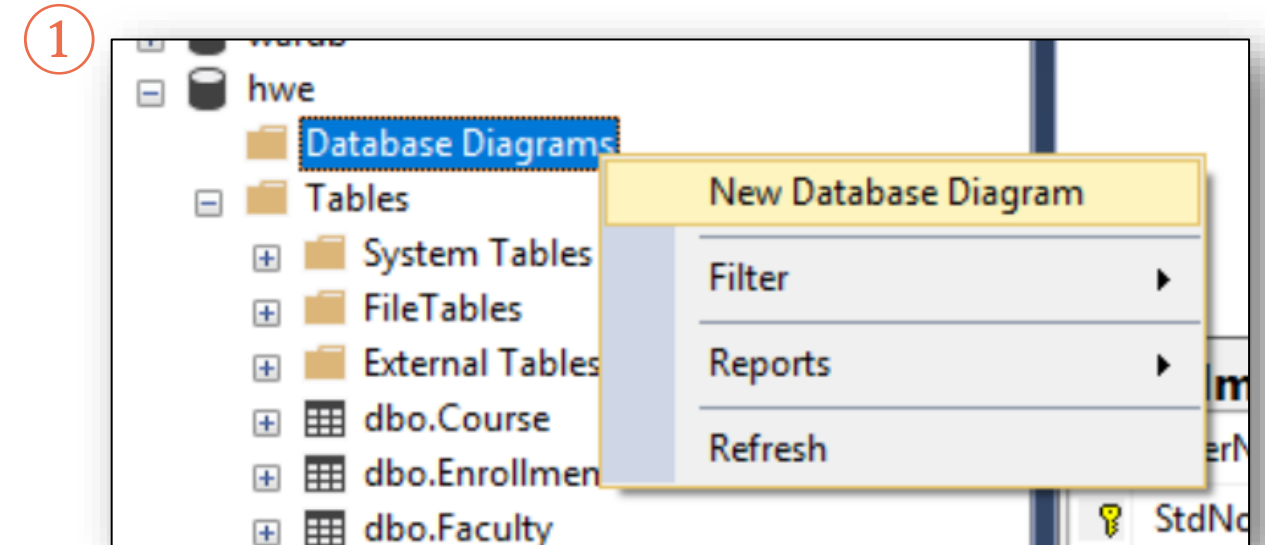


The structure of our little training database (optional)

14 November 2023

Seite 21

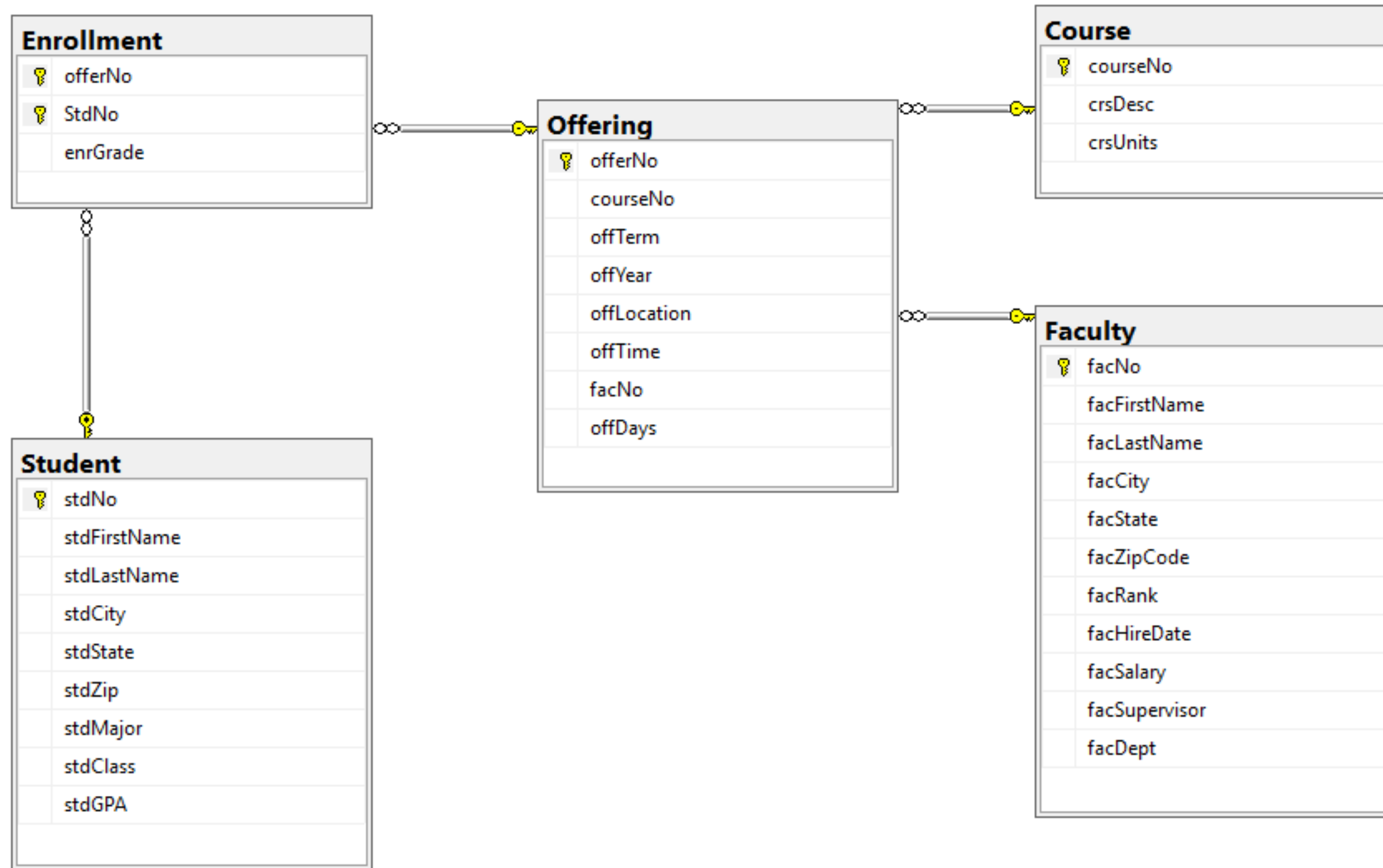
- To view a database Diagramm you need SQL Server Management Studio this feature is not Available in Azure Data Studio
- Select "Database Diagrams"
- Right click and select "New Database Diagram" ①
- Shift-select all tables in "Add Table" ②
- Click 'Add' und then close the window
- Right-click one table-title and select 'Table view' → 'Standard' → What do you see?
- Change the view back to 'Column Names'
- Reposition the tables to get a proper overview (see next page as example)
- Save with a telling name



The Structure of our little training database

14 November 2023

Seite 22



data source and some of the query examples: Michael Mannino, Associate Professor, Business School, University of Colorado Denver

SQL

- Introduction
- Exercises – Setting up the Basis and DDL
- Exercises – DML – Single Table

DML - Data Manipulation Language

- DML consists of all commands to SELECT, change and delete **data** in the database
- The main commands are
 - SELECT
to SELECT and display data
 - INSERT
to add new rows
 - UPDATE
to change data
 - DELETE
to erase rows FROM the table



Examples

```
SELECT * FROM Student;
```

```
INSERT INTO
```

```
    Student( stdNo, stdFirstName, stdLastName, stdCity,  
stdState, stdZip, stdMajor, stdClass, stdGPA)  
    VALUES ( '0987654321', 'Harry', 'Potter', 'London', 'BW',  
'1234-5678', 'IS', 'FR', 4);
```

```
UPDATE Student
```

```
    SET stdCity = 'Little Whinging'  
    WHERE stdNo = '0987654321';
```

```
DELETE FROM Student
```

```
    WHERE stdNo = '0987654321';
```



Exercises – Single table – SELECT 1

SELECT * **FROM** Student;

	stdNo	stdFirstName	stdLastName	stdCity	stdState	stdZip	stdMajor	stdClass	stdGPA
1	123-45-6789	HOMER	WELLS	SEATTLE	WA	98121-1111	IS	FR	3.00
2	124-56-7890	BOB	NORBERT	BOTHELL	WA	98011-2121	FIN	JR	2.70
3	234-56-7890	CANDY	KENDALL	TACOMA	WA	99042-3321	ACCT	JR	3.50
4	345-67-8901	WALLY	KENDALL	SEATTLE	WA	98123-1141	IS	SR	2.80
5	456-78-9012	JOE	ESTRADA	SEATTLE	WA	98121-2333	FIN	SR	3.20
6	567-89-0123	MARIAH	DODGE	SEATTLE	WA	98114-0021	IS	JR	3.60

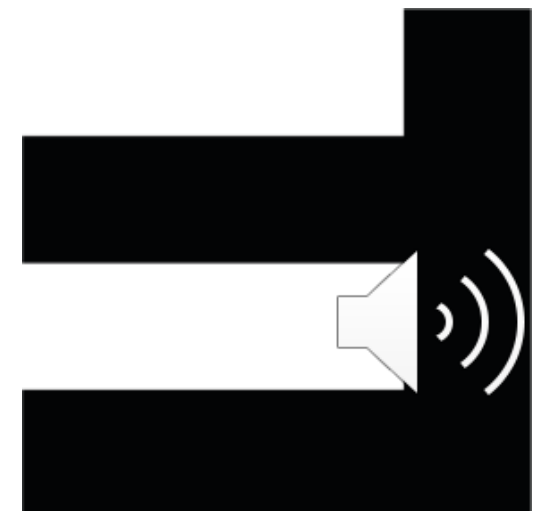
Todo: List the content of the other tables in our database

A more realistic selection:

SELECT * **FROM** Student **WHERE** stdMajor = 'FIN';

Todo: Create other filters,

e.g., find all students from Seattle, or all Profs from Faculty

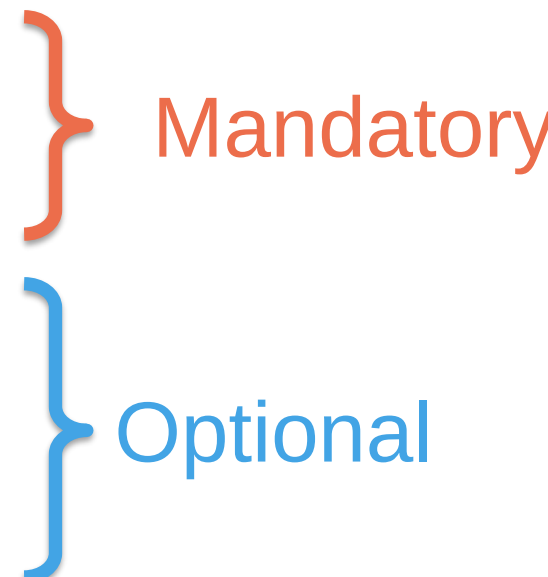


SELECT – some Details

14 November 2023

Seite 27

SELECT <list of column expressions>
FROM <list of tables and join operations>
WHERE <list of logical expressions for rows>
GROUP BY <list of grouping specifications>
ORDER BY <list of sorting specifications>



Mandatory

Optional

SELECT – T-SQL Syntax

14 November 2023

Seite 28

```
<SELECT statement> ::=
    [ WITH { [ XMLNAMESPACES ,] [ <common_table_expression> [,...n] ] } ]
    <query_expression>
    [ ORDER BY { order_by_expression | column_position [ ASC | DESC ] }
    [ ,...n ] ]
    [ <FOR Clause> ]
    [ OPTION ( <query_hint> [ ,...n ] ) ]
<query_expression> ::=
    { <query_specification> | ( <query_expression> ) }
    [ { UNION [ ALL ] | EXCEPT | INTERSECT }
      <query_specification> | ( <query_expression> ) [...n] ]
<query_specification> ::=
SELECT [ ALL | DISTINCT ]
    [ TOP ( expression ) [ PERCENT ] [ WITH TIES ] ]
    < SELECT_list >
    [ INTO new_table ]
    [ FROM { <table_source> } [ ,...n ] ]
    [ WHERE <search_condition> ]
    [ <GROUP BY> ]
    [ HAVING < search_condition > ]
```

Exercises – Single table – SELECT 2

14 November 2023

Seite 29

To add a second condition use AND
(OR is also possible)

```
SELECT * FROM "Student"  
    WHERE "stdMajor" = 'IS' AND "stdClass"='SR';  
SELECT * FROM [Student]  
    WHERE [stdMajor] = 'IS' AND [stdClass]='SR';
```

For schema objects
the quote character
double quote
(where needed)

For strings
use single
quotes

With selected columns and sorting

```
SELECT stdFirstName, stdLastname, stdMajor  
FROM Student  
WHERE stdLastname LIKE '%da%'  
ORDER BY stdLastName asc;
```

Wildcards
_ for one char,
% for multiple
chars

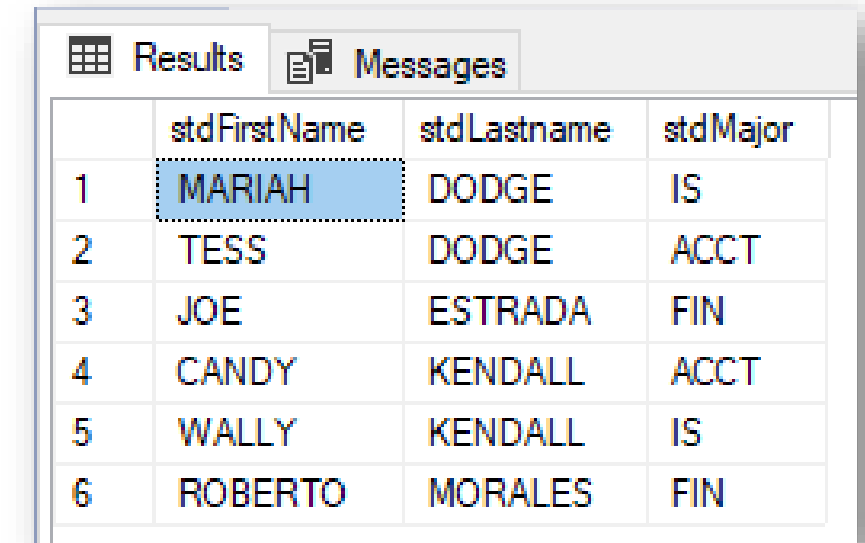
Exercises – Single table – SELECT 3

14 November 2023

Seite 30

- SELECT those students with last name between D and M (FROM Daaaaa to Mzzzzz)

```
SELECT stdFirstName, stdLastname, stdMajor  
FROM Student  
WHERE stdLastName > 'D'  
      AND stdLastName < 'N'  
ORDER BY stdLastName;
```



	stdFirstName	stdLastname	stdMajor
1	MARIAH	DODGE	IS
2	TESS	DODGE	ACCT
3	JOE	ESTRADA	FIN
4	CANDY	KENDALL	ACCT
5	WALLY	KENDALL	IS
6	ROBERTO	MORALES	FIN

alternative:

```
WHERE stdLastName  
      BETWEEN 'D'  
      AND 'N'
```

Exercises – Single table – SELECT 4

14 November 2023

Page 31

- SELECT those professors with salary equal or lower than 70000

```
SELECT * FROM Faculty  
WHERE FacRank= 'PROF'  
AND facSalary <= 70000;
```

	facNo	facFirstName	facLastName	facCity	facState	facZipCode	facRank	facHireDate	facSalary	facSupervisor	facDept
1	765-43-2109	NICKI	MACON	BELLEVUE	WA	98015-9945	PROF	2002-04-11	65000.00		FIN

- List the hometowns of the students

```
SELECT DISTINCT stdCity as 'City'  
FROM Student;
```

	stdCity
1	SEATTLE
2	BOTHELL
3	TACOMA
4	SEATTLE
5	SEATTLE
6	SEATTLE
7	REDMOND
8	SEATTLE
9	SEATTLE
10	SEATTLE
11	BOTHELL

	City
1	BOTHELL
2	REDMOND
3	SEATTLE
4	TACOMA

Exercises – Single table – SELECT 5

14 November 2023

Seite 32

- Defining the list titles

```
SELECT facFirstName AS 'First Name',  
       facLastName AS 'Last Name'  
FROM Faculty;
```

facFirstName	facLastName
LEONARD	VINCE
VICTORIA	EMMANUEL
LEONARD	FIBON
NICKI	MACON
CRISTOPHER	COLAN
JULIA	MILLS

First Name	Last Name
LEONARD	VINCE
VICTORIA	EMMANUEL
LEONARD	FIBON
NICKI	MACON
CRISTOPHER	COLAN
JULIA	MILLS

Exercise Time:

1. Simple query statements

Please do the exercises in: `Exercise1.ipynb` and `Exercise2a.ipynb`

Exercises – Single table – SELECT 6

14 November 2023

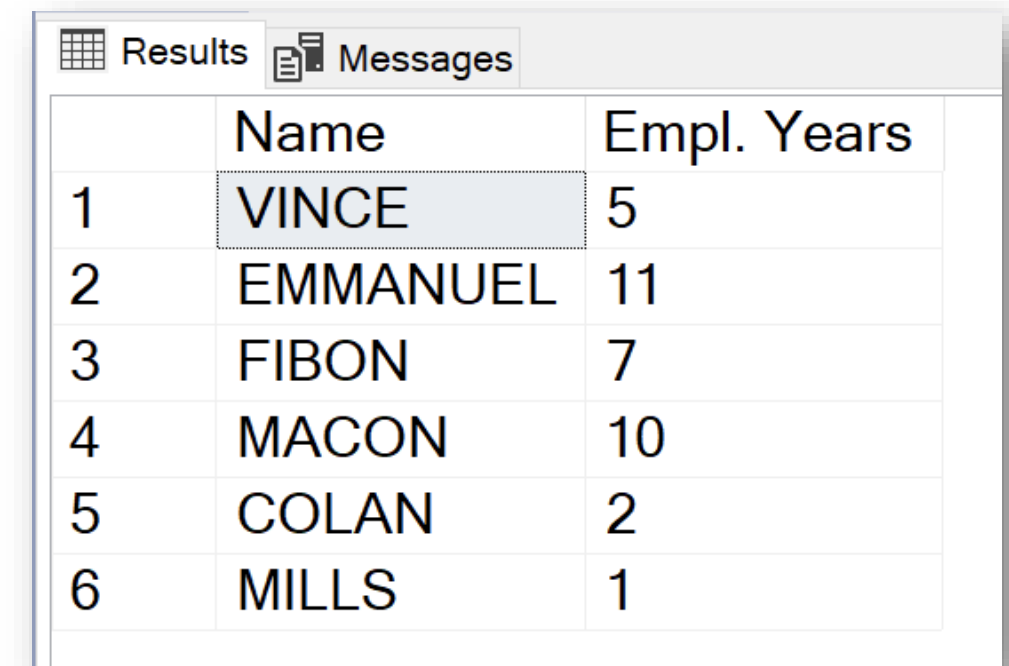
Seite 34

- Calculating Years

```
SELECT facLastName AS 'Name',  
       YEAR(GETDATE()) -  
       YEAR(FacHireDate)  
       AS 'Empl. Years'  
FROM Faculty;
```

- Variant

```
SELECT facLastName AS 'Name',  
       DATEPART(YYYY, getdate()) -  
       DATEPART(YYYY, FacHireDate)  
       AS 'Empl. Years'  
FROM Faculty;
```



	Name	Empl. Years
1	VINCE	5
2	EMMANUEL	11
3	FIBON	7
4	MACON	10
5	COLAN	2
6	MILLS	1

Research for
GETDATE and YEAR
in the Transact-SQL
documentation

Exercises – Single table – SELECT 6

- Calculating years of employment

The solutions on the previous slide ignore the exact date
The full solution is more complex – here just provided for information and NOT part of the “need-to-know”.

```
SELECT facLastName AS 'Name',  
CASE WHEN DATEADD(YEAR, DATEDIFF (YEAR, facHireDate,  
GETDATE()), FACHIREDATE) > GETDATE()  
THEN DATEDIFF(YEAR, facHireDate, GETDATE()) - 1  
ELSE DATEDIFF(YEAR, facHireDate, GETDATE())  
END  
AS 'Empl. Years' , facHireDate  
FROM Faculty;
```

	Name	Empl. Years	facHireDate
1	VINCE	5	2017-04-10
2	EMMANUEL	11	2011-04-15
3	FIBON	6	2015-12-01
4	MACON	9	2012-11-11
5	COLAN	2	2020-03-01
6	MILLS	1	2021-03-15



Exercises – Single table – SELECT 7

14 November 2023

Seite 36

- Show the students studying IS ordered by name with a GPA 1 point better (=higher) than it really is (Table Student)

SELECT

```
stdFirstName,  
stdLastName,  
stdGPA,  
stdGPA +1 AS 'new GPA'
```

FROM

Student

WHERE

```
stdMajor = 'IS'
```

ORDER BY

```
stdLastName,  
stdFirstname;
```

stdFirstName	stdLastName	stdGPA	new GPA
LUKE	BRAZZI	2.20	3.20
CRISTOPHER	COLAN	4.00	5.00
MARIAH	DODGE	3.60	4.60
WALLY	KENDALL	2.80	3.80
WILLIAM	PILGRIM	3.80	4.80
HOMER	WELLS	3.00	4.00

Exercises – Single table – SELECT 7b

- Now, let's do it. Change the GPA for the IS students. Add a point to their grades.

- UPDATE** Student

```
SET stdGPA = stdGPA + 1
```

```
WHERE stdMajor = 'IS';
```

```
-- to show the results
```

```
SELECT
```

```
stdFirstName, stdLastName, stdGPA
```

```
FROM Student
```

```
WHERE stdMajor = 'IS'
```

```
ORDER BY stdLastName, stdFirstname;
```

	stdFirstName	stdLastName	stdGPA
1	LUKE	BRAZZI	3.20
2	CRISTOPHER	COLAN	5.00
3	MARIAH	DODGE	4.60
4	WALLY	KENDALL	3.80
5	WILLIAM	PILGRIM	4.80
6	HOMER	WELLS	4.00



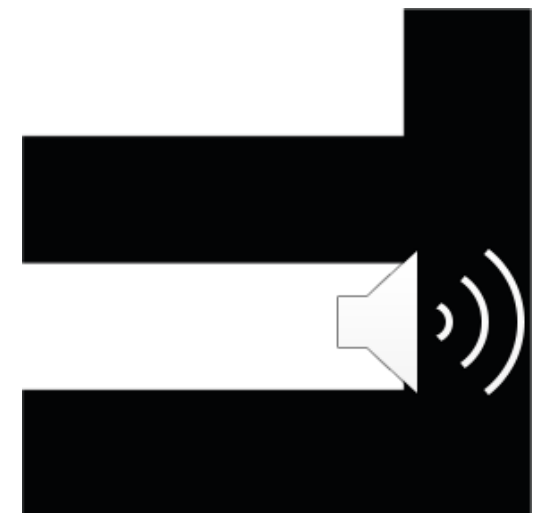
Aggregates

An aggregate function performs a calculation on a set of values and returns a single value.

Aggregate functions are often used with the GROUP BY clause of the SELECT statement.

Examples:

- **COUNT** (<expression>) – Counts the elements
- **MIN** (<expression>) – Least value of <expression>
- **MAX** (<expression>) – Greatest value of <expression>
- **SUM** (<expression>) – Sum of all defined values of <expression>
- **AVG** (<expression>) – Unweighted average of all defined values
- **VAR** (<expression>) – Variance of all defined values
- **STDDEV** (<expression>) – Standard deviation of all defined values



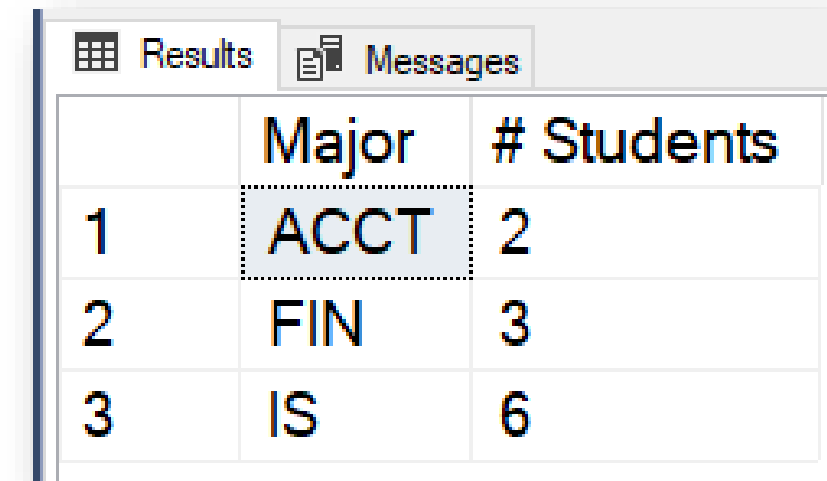
Exercises – Single table – SELECT 8

14 November 2023

Seite 39

- Grouping and counting (Table Student; Field: stdMajor)

```
SELECT stdMajor AS 'Major',  
       COUNT(*) AS '# Students'  
FROM Student  
GROUP BY stdMajor;
```

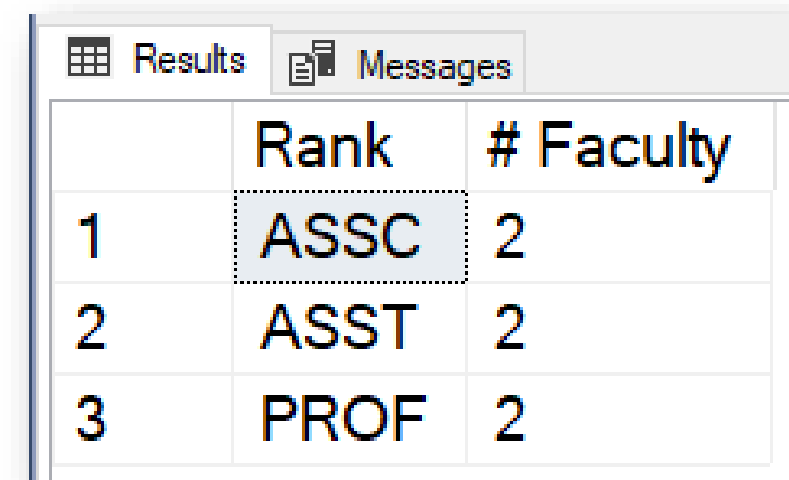


The screenshot shows a database query results window with two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with three columns: an index, 'Major', and '# Students'. The data is as follows:

	Major	# Students
1	ACCT	2
2	FIN	3
3	IS	6

- Similar task for the table Faculty with Rank as the grouping element
(Table Faculty; Field: facRank)

```
SELECT facRank AS 'Rank',  
       COUNT(*) AS '# Faculty'  
FROM Faculty  
GROUP BY facRank;
```



The screenshot shows a database query results window with two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with three columns: an index, 'Rank', and '# Faculty'. The data is as follows:

	Rank	# Faculty
1	ASSC	2
2	ASST	2
3	PROF	2

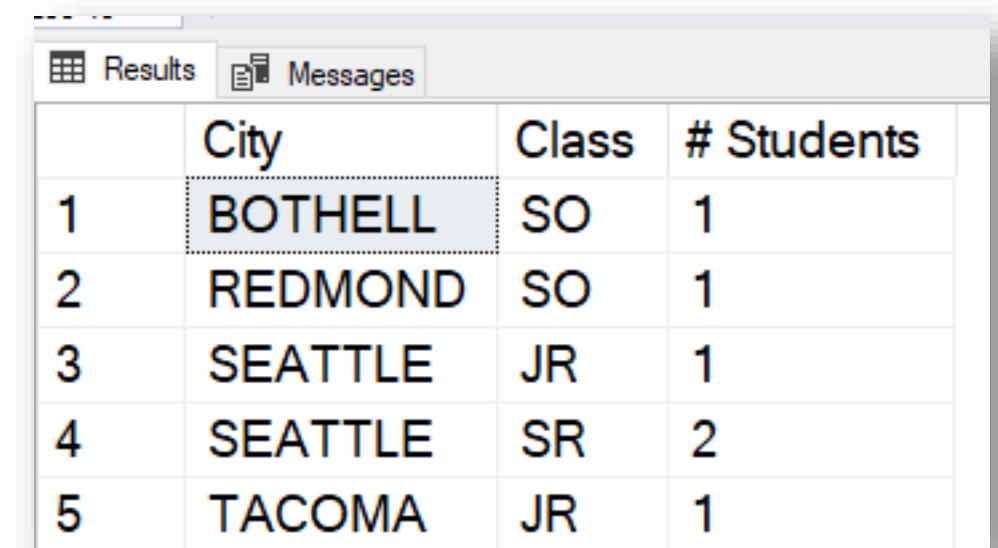
Exercises – Single table – SELECT 8c

14 November 2023

Seite 40

- Give me the number of students from each city and class with a GPA better than 3 (Table Student; Fields: stdCity, stdClass, stdGPA)

```
SELECT stdCity AS 'City', stdClass AS 'Class',  
COUNT(*) AS '# Students'  
FROM Student  
WHERE stdGPA > 3  
GROUP BY stdCity, stdClass  
ORDER BY stdCity;
```



	City	Class	# Students
1	BOTHELL	SO	1
2	REDMOND	SO	1
3	SEATTLE	JR	1
4	SEATTLE	SR	2
5	TACOMA	JR	1

Exercises – Single table – SELECT 9

14 November 2023

Seite 41

- What is the employees' average, minimum and maximum salary by rank? (Table Faculty, Fields: facSalary)

Rank	Average	Minimum	Maximum
ASSC	72500.00	70000.00	75000.00
ASST	37500.00	35000.00	40000.00
PROF	92500.00	65000.00	120000.00

```
SELECT facRank AS 'Rank',  
       FORMAT(AVG(facSalary) , 'N2') AS 'Average',  
       MIN(facSalary) AS 'Minimum',  
       MAX(facSalary) AS 'Maximum'  
FROM Faculty  
GROUP BY facRank;
```

More details to number formatting: <https://docs.microsoft.com/en-us/dotnet/standard/base-types/standard-numeric-format-strings>

Exercises – Single table – SELECT 9

14 November 2023

Seite 42

- Examples for some variants of the format function

```
SELECT facRank AS 'Rank',  
--  AVG(facSalary),  
    ROUND (AVG(facSalary) , 2) AS 'Average',  
    FORMAT(AVG(facSalary) , 'F3', 'en-us') AS 'Average F2',  
    FORMAT(AVG(facSalary) , 'N0', 'en-us') AS 'Average N2',  
    FORMAT(AVG(facSalary) , 'C2', 'en-us') AS 'Average C2',  
    FORMAT(AVG(facSalary) , 'C2', 'de-de') AS 'Average C2de',  
    MIN(facSalary) AS 'Minimum',  
    MAX(facSalary) AS 'Maximum'  
FROM Faculty  
GROUP BY facRank;
```

Filtering Groups

- Groups (these are fields where aggregate functions are used on) are filtered by using the HAVING statement filters.
- The HAVING statement comes behind the WHERE statement (if available)
- HAVING is always used against the aggregate function, not the field name

```
SELECT CustomerID, COUNT(*) AS Count_Orders  
FROM Sales.SalesOrderHeader  
GROUP BY CustomerID  
HAVING COUNT(*) > 10;
```



Order of SQL-Statements being processed

14 November 2023

Seite 44

Logical Order	Phase	Comments
5	SELECT	
1	FROM	
2	WHERE	
3	GROUP BY	Creates groups
4	HAVING	Operates on groups
6	ORDER BY	

Example for a complete statement with everything we did so far:

```
SELECT CustomerID, COUNT(*) AS Count_Orders
FROM Sales.SalesOrderHeader
WHERE YEAR(OrderDate) > 2013
GROUP BY CustomerID
HAVING COUNT(*) > 10
ORDER BY Count_Orders;
```



Exercise Time:

2. Aggregate queries

Please do the exercises in: Exercise2.ipynb

Hier geht es weiter

Techn. Info.

Systems

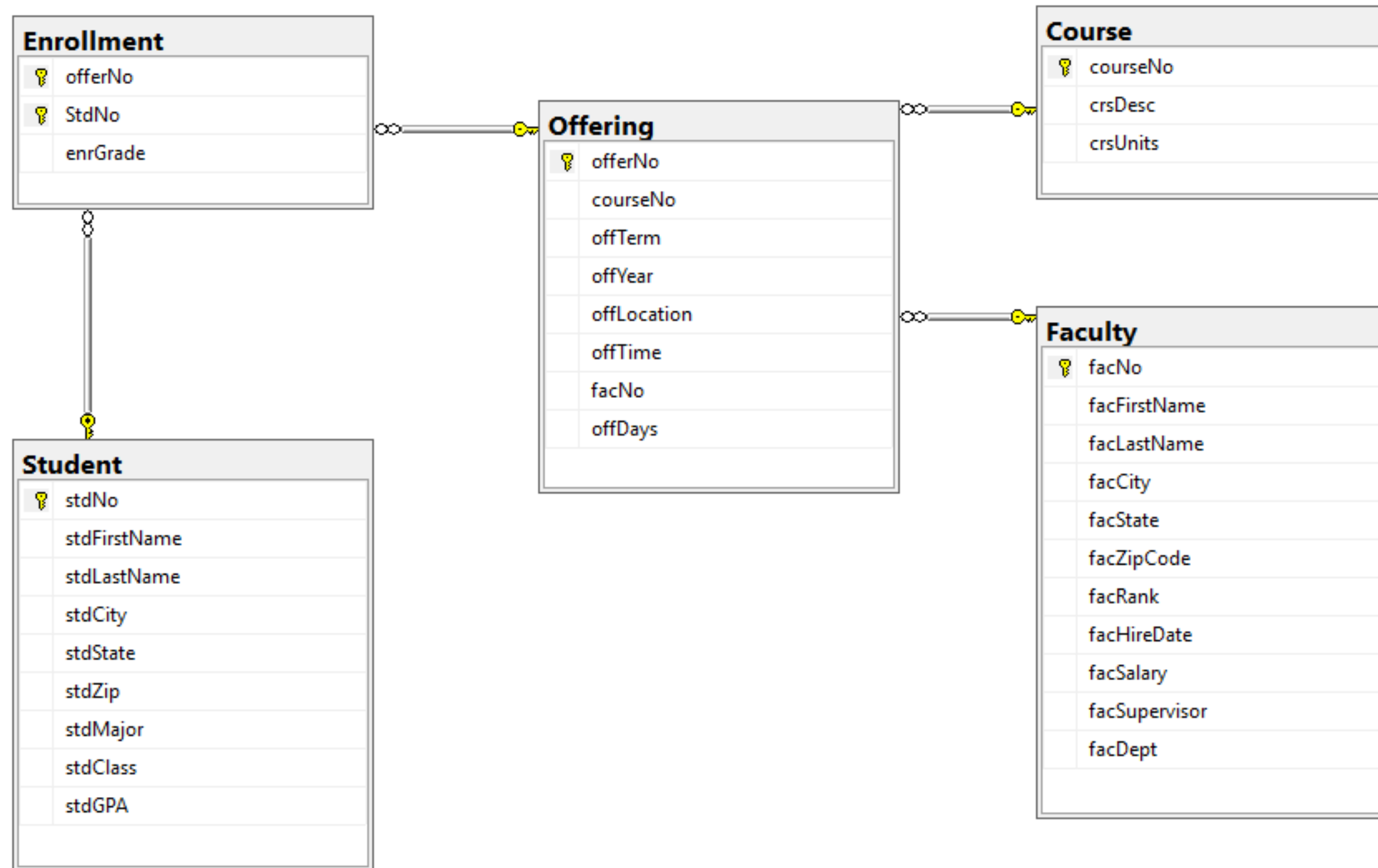
SQL

- Introduction
- Exercises – Setting up the Basis and DDL
- Exercises – DML – Single Table
- Exercises – DML – Joining Multiple Tables

Our little training database

14 November 2023

Seite 48



data source and some of the query examples: Michael Mannino, Associate Professor, Business School, University of Colorado Denver

Combining multiple tables

14 November 2023

Seite 49

- The simplest combination of two tables is the Cartesian product.
- The Cartesian product takes all tuples of one table and combines them with all tuples of the second table

T1	T1_PK	T1_more
	1	hugo
	2	otto
	3	fred

T2	T2_name	T2_FK
	sue	3
	nina	2
	pen	3

T1xT2 →

```
SELECT * FROM T1, T2;  
SELECT * FROM T1 CROSS JOIN T2;
```

T1_PK	T1_more	T2_name	T2_FK
1	hugo	sue	3
1	hugo	nina	2
1	hugo	pen	3
2	otto	sue	3
2	otto	nina	2
2	otto	pen	3
3	fred	sue	3
3	fred	nina	2
3	fred	pen	3

Combining with WHERE

- Equi Join → To get reasonable results

```
SELECT * FROM T1, T2 WHERE T1_PK=T2_FK;
```

T1_PK	T1_more	T2_name	T2_FK
1	hugo	sue	3
1	hugo	nina	2
1	hugo	pen	3
2	otto	sue	3
2	otto	nina	2
2	otto	pen	3
3	fred	sue	3
3	fred	nina	2
3	fred	pen	3

T1_PK	T1_more	T2_name	T2_FK
2	otto	nina	2
3	fred	sue	3
3	fred	pen	3

INNER JOIN

14 November 2023

Seite 51

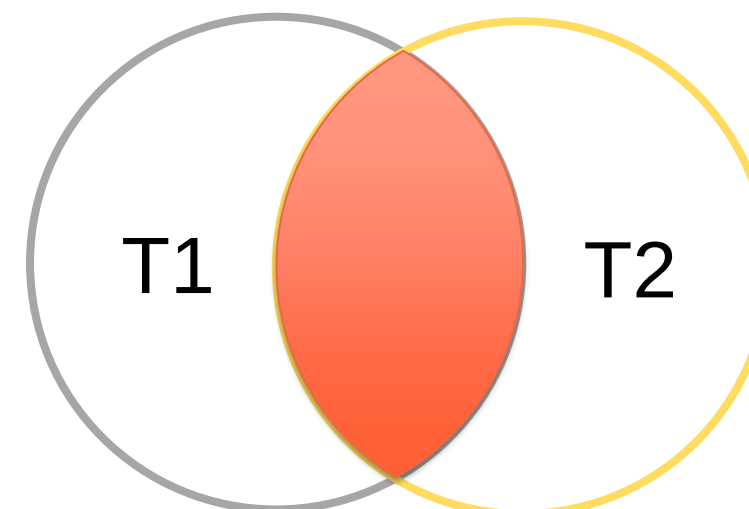
```
SELECT * FROM T1, T2 WHERE T1_PK=T2_FK;
```

- The same with a JOIN Clause

```
SELECT * FROM T1 INNER JOIN T2 ON T1_PK=T2_FK;
```

T1_PK	T1_more	T2_name	T2_FK
1	hugo	sue	3
1	hugo	nina	2
1	hugo	pen	3
2	otto	sue	3
2	otto	nina	2
2	otto	pen	3
3	fred	sue	3
3	fred	nina	2
3	fred	pen	3

T1_PK	T1_more	T2_name	T2_FK
2	otto	nina	2
3	fred	sue	3
3	fred	pen	3



Exercises – Multiple tables – SELECT 1a

14 November 2023

Seite 52

- SELECT the course offerings with no assigned lecturer (facNo in Table Offering) and the full Course Name

```
SELECT crsDesc AS 'Course Name',  
       OffTerm AS 'Term', OffYear AS 'Year'  
FROM Offering INNER JOIN Course  
ON Course.CourseNo = Offering.CourseNo;
```

What do we get here?

Results Messages			
	Course Name	Term	Year
1	FUNDAMENTALS OF BUSINESS PROGRAMMING	SUMMER	2020
2	FUNDAMENTALS OF BUSINESS PROGRAMMING	FALL	2019
3	SYSTEMS ANALYSIS	SUMMER	2019
4	FUNDAMENTALS OF BUSINESS PROGRAMMING	SPRING	2020
5	FUNDAMENTALS OF BUSINESS PROGRAMMING	FALL	2019
6	FUNDAMENTALS OF BUSINESS PROGRAMMING	WINTER	2020

Required
Tables:
Offering
Course

Exercises – Multiple tables – SELECT 1b

14 November 2023

Seite 53

- SELECT the course offerings with no assigned lecturer

```
SELECT *  
FROM Offering  
WHERE FacNo IS NULL;
```

offerNo	courseNo	offTerm	offYear	offLocation	offTime	facNo	offDays
1111	IS320	SUMMER	2020	BLM302	10:30:00	NULL	MW
2222	IS460	SUMMER	2019	BLM412	13:30:00	NULL	TTH



Course Name	Semester
FUNDAMENTALS OF BUSINESS PROGRAMMING	SUMMER 2020
SYSTEMS ANALYSIS	SUMMER 2019

Exercises – Multiple tables – SELECT 1b

14 November 2023

Seite 54

- We need the course name to make the list more legible (use JOIN)

```
SELECT crsDesc AS 'Course Name',  
       OffTerm, OffYear  
FROM Course INNER JOIN Offering  
  ON Course.CourseNo = Offering.CourseNo  
WHERE FacNo IS NULL;
```

- important Shortcut
Alias:

```
FROM Course c INNER JOIN Offering o  
  ON c.CourseNo = o.CourseNo
```

Required
Tables:
Offering
Course

Exercises – Multiple tables – SELECT 2

14 November 2023

Seite 55

- 1) Find for each offering the number of students assigned to it.
- 2) Enrich the data with the Fields offTerm, offYear, CourseNo.

```
SELECT offerNo, COUNT(*) AS 'Student #'  
FROM Enrollment  
GROUP By offerNo;
```

	offerNo	Student #
1	1234	6
2	4321	6
3	5555	2
4	5678	5
5	5679	6
6	6666	2
7	7777	3
8	9876	7



	offerNo	offTerm	offYear	CourseNo	Student #
1	1234	FALL	2019	IS320	6
2	4321	FALL	2019	IS320	6
3	5555	WINTER	2019	FIN300	2
4	5678	WINTER	2020	IS480	5
5	5679	SPRING	2020	IS480	6
6	6666	WINTER	2020	FIN450	2
7	7777	SPRING	2020	FIN480	3
8	9876	SPRING	2020	IS460	7

Required
Tables:
Offering
Enrollment

Exercises – Multiple tables – SELECT 2

14 November 2023

Seite 56

- Connect the tables

```
SELECT offerNo, COUNT(*) AS 'Student #'  
FROM Enrollment  
GROUP By offerNo;
```



```
SELECT o.offerNo, COUNT(*) AS 'Student #'  
FROM Offering o INNER JOIN Enrollment e  
    ON o.offerNo=e.offerNo  
GROUP BY o.offerNo;
```

Required
Tables:
Offering
Enrollment

Exercises – Multiple tables – SELECT 2

14 November 2023

Seite 57

- Add the attributes offerNo, offTerm, offYear, CourseNo

```
SELECT o.offerNo, COUNT(*) AS 'Student #'  
FROM Offering o INNER JOIN Enrollment e  
  ON o.offerNo=e.offerNo  
GROUP BY o.offerNo;
```



```
SELECT o.offerNo, o.offTerm, o.offYear, o.CourseNo,  
  COUNT(*) AS 'Student #'  
FROM Offering o INNER JOIN Enrollment e  
  ON o.offerNo=e.offerNo  
GROUP BY o.offerNo, o.offTerm, o.offYear, o.courseNo;
```

Next: Extending the SELECT statement to get the course description

Exercises – Multiple tables – SELECT 3

14 November 2023

Seite 58

- Extending the SELECT statement to get the course description

```
SELECT e.offerNo, o.offTerm, o.offYear, c.crsDesc AS 'Course',  
       COUNT(*) AS 'Student #'  
FROM Offering o INNER JOIN Enrollment e  
  ON o.offerNo=e.offerNo  
   INNER JOIN Course c  
  ON o.courseNo=c.courseNo  
GROUP BY e.offerNo, o.offTerm, o.offYear, c.crsDesc;
```

Required
Tables:
Offering
Enrollment
Course

	offerNo	offTerm	offYear	Course	Student #
1	1234	FALL	2019	FUNDAMENTALS OF BUSINESS PROGRAMMING	6
2	4321	FALL	2019	FUNDAMENTALS OF BUSINESS PROGRAMMING	6
3	5555	WINTER	2019	FUNDAMENTALS OF FINANCE	2
4	5678	WINTER	2020	FUNDAMENTALS OF DATABASE MANAGEMENT	5
5	5679	SPRING	2020	FUNDAMENTALS OF DATABASE MANAGEMENT	6
6	6666	WINTER	2020	PRINCIPLES OF INVESTMENTS	2

Exercises – Multiple tables – SELECT 4

14 November 2023

Seite 59

- SELECT the offerings with course descriptions, faculty names, and schedule info (use **CONCAT()** to combine the attributes in one column)

```
SELECT c.crsDesc AS 'Course',  
       CONCAT(f.facFirstname, ' ', f.facLastName) AS 'Teacher',  
       CONCAT(o.offTerm, o.offYear, ' ', o.offDays) AS 'Time'  
FROM Offering o INNER JOIN Course c  
ON o.courseNo=c.courseNo  
INNER Join Faculty f  
ON f.facNo=o.facNo  
ORDER BY o.offYear, o.offTerm, o.offDays;
```

Required
Tables:
Offering
Course
Faculty

	Course	Teacher	Time
1	FUNDAMENTALS OF BUSINESS PROGRAMMING	LEONARD VINCE	FALL 2019 MW
2	FUNDAMENTALS OF BUSINESS PROGRAMMING	LEONARD VINCE	FALL 2019 TTH
3	FUNDAMENTALS OF FINANCE	NICKI MACON	WINTER2019 MW
4	FUNDAMENTALS OF BUSINESS PROGRAMMING	LEONARD VINCE	SPRING2020 MW

JOIN expressions compared

- INNER JOIN gives you all records that include the same values in the compared fields

```
SELECT * FROM Course c INNER JOIN Offering o  
ON c.courseNo=c.courseNo;
```

Is equivalent to

```
SELECT * FROM Course c, Offering o  
WHERE c.courseNo=c.courseNo;
```

JOIN expressions compared

- So the following expression is equivalent to the expression used in the last exercise:

```
SELECT c.crsDesc AS 'Course',  
       CONCAT(f.facFirstname, ' ', f.facLastName) AS 'Teacher',  
       CONCAT(o.offTerm, o.offYear, ' ', o.offDays) AS 'Time'  
FROM Offering o, Course c, Faculty f  
WHERE f.facNo=o.facNo  
      AND c.courseNo=o.courseNo  
ORDER BY o.offYear, o.offTerm, o.offDays;
```

Exercise Time:

3. Joins between tables

Please do the exercises in: `Exercis3.ipynb`

SQL

- Introduction
- Exercises – Setting up the Basis and DDL
- Exercises – DML – Single Table
- Exercises – DML – Joining Multiple Tables
- Exercises – DML – Advanced Joins

OUTER JOIN

- Preparation of our data
Change

UPDATE Offering

SET facNo= '098-76-5432'

WHERE facNo= '876-54-3210' ;

- Be careful – you change all records that match the WHERE clause.
Without WHERE clause you even change all records of the table!

OUTER JOIN

- The OUTER JOIN lists also records with no reference in the corresponding table.
- FULL OUTER JOIN uses the rule for both tables. RIGHT and LEFT OUTER JOIN restricts the selection to the respective table.

- Try and analyse

```
SELECT * FROM Offering o FULL OUTER JOIN Faculty f
    ON o.facNo=f.facNo;
```

and

```
SELECT * FROM Offering o LEFT OUTER JOIN Faculty f
    ON o.facNo=f.facNo;
```

and

```
SELECT * FROM Offering o RIGHT OUTER JOIN Faculty f
    ON o.facNo=f.facNo;
```

OUTER JOIN

```
SELECT *  
FROM Offering;
```

Left

offerNo	courseNo	offTerm	offYear	offLocation	offTime	facNo	offDays
1111	IS320	SUMMER	2013	BLM302	10:30:00	NULL	MW
1234	IS320	FALL	2012	BLM302	10:30:00	098-76-5432	TW
2222	IS460	SUMMER	2012	BLM412	13:30:00	NULL	TTH
3333	IS320	SPRING	2013	BLM214	08:30:00	098-76-5432	MW
4321	IS320	FALL	2012	BLM214	15:30:00	098-76-5432	TTH
4444	IS320	WINTER	2013	BLM302	15:30:00	543-21-0987	TTH
5555	FIN300	WINTER	2012	BLM207	08:30:00	765-43-2109	MW
5678	IS480	WINTER	2013	BLM302	10:30:00	987-65-4321	MW
5679	IS480	SPRING	2013	BLM412	15:30:00	NULL	TTH
6666	FIN450	WINTER	2013	BLM212	10:30:00	987-65-4321	TTH
7777	FIN480	SPRING	2013	BLM305	13:30:00	765-43-2109	MW
8888	IS320	SUMMER	2013	BLM405	13:30:00	654-32-1098	MW
9876	IS460	SPRING	2013	BLM307	13:30:00	654-32-1098	TTH

```
SELECT *  
FROM Faculty;
```

Right

facNo	facFirstName	facLastName	facCity	facSta
098-76-5432	LEONARD	VINCE	SEATTLE	WA
543-21-0987	VICTORIA	EMMANUEL	BOTHELL	WA
654-32-1098	LEONARD	FIBON	SEATTLE	WA
765-43-2109	NICKI	MACON	BELLEVUE	WA
876-54-3210	CRISTOPHER	COLAN	SEATTLE	WA
987-65-4321	JULIA	MILLS	SEATTLE	WA

?

Exercises – Multiple tables – SELECT 4

– With OUTER JOIN

14 November 2023

Seite 67

- The Result of the last exercise WITH the offerings that do not have a lecturer

```
SELECT c.crsDesc AS 'Course',  
       CONCAT(f.facFirstname, ' ', f.facLastName) AS 'Teacher',  
       CONCAT(o.offTerm, o.offYear, ' ', o.offDays) AS 'Time'  
FROM Offering o INNER JOIN Course c  
ON o.courseNo=c.courseNo  
LEFT OUTER JOIN Faculty f  
ON f.facNo=o.facNo  
ORDER BY o.offYear, o.offTerm, o.offDays;
```



2	FUNDAMENTALS OF BUSINESS PROGRAMMING	LEONARD VINCE	FALL 2012 TTH
3	SYSTEMS ANALYSIS		SUMMER2012 TTH
4	FUNDAMENTALS OF FINANCE	NICKI MACON	WINTER2012 MW
8	FUNDAMENTALS OF DATABASE MANAGEMENT	CRISTOPHER COLAN	SPRING2013 TTH
9	FUNDAMENTALS OF BUSINESS PROGRAMMING		SUMMER2013 MW
10	FUNDAMENTALS OF BUSINESS PROGRAMMING	LEONARD VINCE	SUMMER2013 MW

Exercises – Multiple tables – SELECT 5

14 November 2023

Seite 68

- Add your name to the student list using the edit function of the interface
- List all students with the number of their enrollments

```
SELECT (stdFirstName + ' ' + stdLastName), COUNT(offerNo)
FROM Enrollment e INNER JOIN Student s
ON s.stdNo=e.stdNo
GROUP BY (stdFirstName + ' ' + stdLastName);
```

- Include students without an enrollment

```
SELECT (stdFirstName + ' ' + stdLastName), COUNT(offerNo)
FROM Enrollment e RIGHT OUTER JOIN Student s
ON s.stdNo=e.stdNo
GROUP BY (stdFirstName + ' ' + stdLastName)
ORDER BY COUNT(offerNo);
```

	(No column name)	(No column name)
1	BOB NORBERT	4
2	CANDY KENDALL	4
3	CRISTOPHER COLAN	2
4	HOMER WELLS	5
5	JOE ESTRADA	3

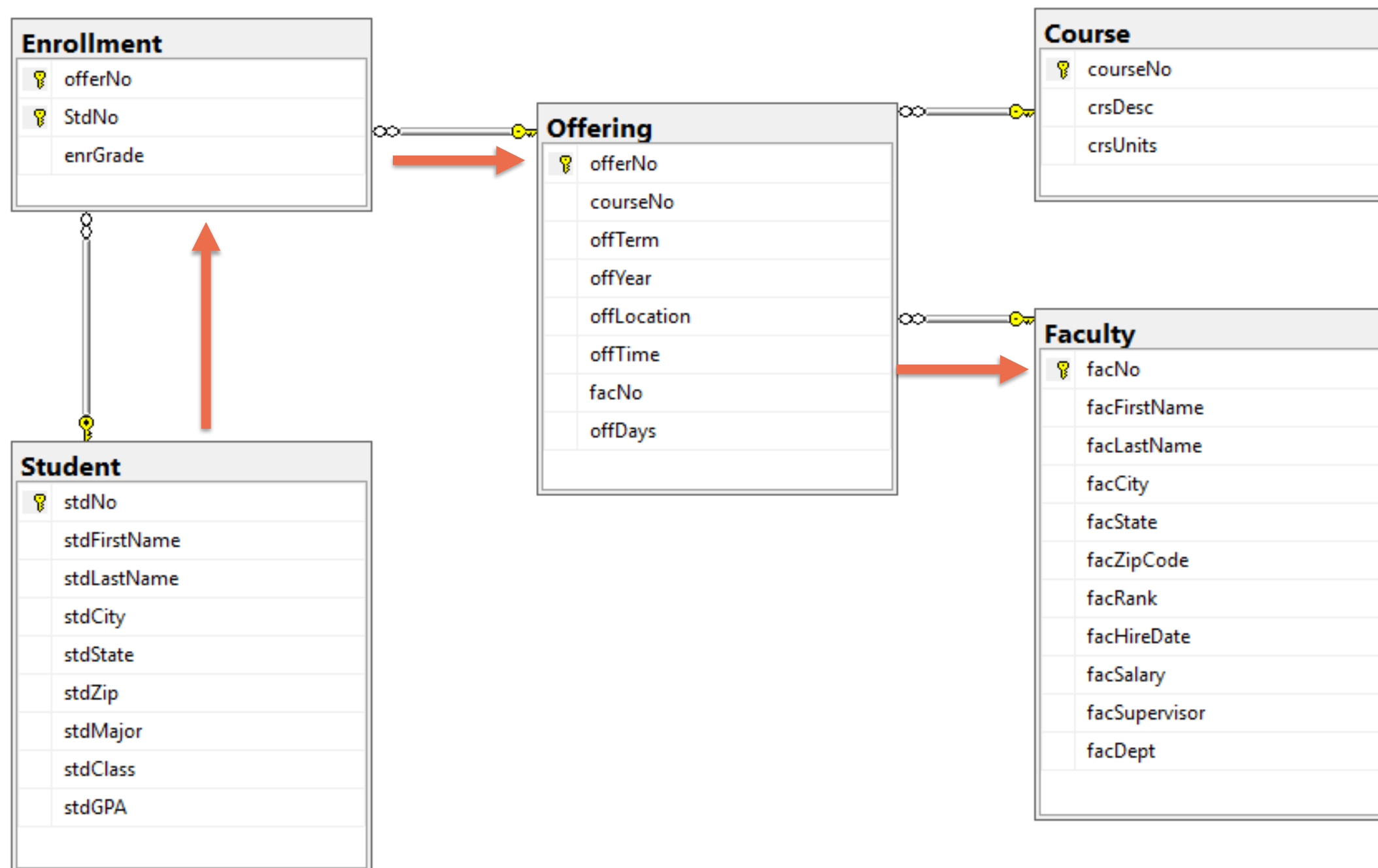
	(No column name)	(No column name)
1	Heidi Weber	0
2	ROBERTO MORALES	2
3	CRISTOPHER COLAN	2
4	JOE ESTRADA	3
5	LUKE BRAZZI	3

Exercises – Multiple tables – SELECT 6

14 November 2023

Seite 69

- List all students with the lecturers of their classes



Exercises – Multiple tables – SELECT 6

14 November 2023

Seite 70

- List all students with the lecturers of their classes

Extend the code as follows (one after the other)

1. Don't allow for repeated rows
2. Sort by student name and lecturer name
3. Count the classes a student has with a lecturer
4. Sort by number of classes held together starting with the highest number

Exercises – Multiple tables

14 November 2023

Seite 71

- Create other combinations and analyse them
- e.g.
 1. Join Students and Enrolment including students without classes
 2. Extend the query so that you can see the number of enrolments
 3. Group them for each student by semester